Text and Sequence Analytics

Compiled Revision Notes

Table of Contents

Week 1: Introduction to Text & Sequence Analytics **4**
- Fundamentals of Text Analytics . . . 4
- The DIKW Pyramid . . . 4
- Artificial Intelligence: Capabilities & Branches . . . 4
- Sequence Analysis & Bioinformatics . . . 5
- Historical Context & Thought Experiments . . . 5
- Modern Tools & Frameworks . . . 5

Week 2: Text Processing & Normalization **5**
- Text Representation . . . 5
- Regular Expressions (Regex) . . . 6
- Text Normalization . . . 6
- Key Stemming Algorithms . . . 7
- Part of Speech (POS) Tagging . . . 7
- Performance Metrics & Data Structures . . . 7

Week 3: Tokenization & Sequence Segmentation **7**
- Text Tokenization Basics . . . 7
- Sub-Word Tokenization . . . 8
- Sequence Segmentation . . . 9
- SentencePiece Framework . . . 10

Week 4: Sequence Similarity & Alignment **11**
- Mathematical Fundamentals of Similarity . . . 11
- Distance Calculations . . . 11
- Sequence Alignment . . . 12
- Advanced Scoring Parameters . . . 13
- BLAST Statistical Significance . . . 13

Week 5: Alignment-Free Sequence Similarity **14**
- Fundamentals of Alignment-Free Similarity . . . 14
- Measures vs. Metrics . . . 14
- Indices, Coefficients, and Distances . . . 14
- Vector-Based Distances . . . 14
- Set-Based Similarities . . . 16
- MinHash Algorithm . . . 18

Week 6: Text Feature Engineering **18**
- Feature Engineering Fundamentals . . . 18
- Challenges of Using Text . . . 18
- Data Sets and Data Frames . . . 19
- Text Encoding and Vectorization . . . 19
- Term Frequency-Inverse Document Frequency . . . 19
- Okapi BM25 . . . 20

Feature Selection and Hashing . . . 21
Dimensionality Reduction . . . 21

Week 7: Introduction to Text Classification **21**
Fundamentals of Text Classification . . . 21
Supervised Classification Models . . . 22
Neural Networks (NN) . . . 23
Activation Functions . . . 23
Data Normalization . . . 24
Gradient Descent & Accelerated Learning . . . 24
Network Topology . . . 25
Unsupervised Learning (Clustering) . . . 25

Week 8: Introduction to Word Embeddings **26**
Fundamentals of Word Embeddings . . . 26
Static vs. Context-Sensitive Embeddings . . . 26
Geometric Properties & Word Analogies . . . 26
One-Hot Encoding vs. Dense Embeddings . . . 27
Word2Vec Architecture . . . 27

Week 9: Word2Vec Methods & Skip-Gram vs. CBOW **28**
Continuous Bag of Words (CBOW) . . . 28
Skip-Gram . . . 28
Mathematical Objective: Softmax and Cross-Entropy . . . 29
Training Optimizations . . . 29
Skip-Gram vs. CBOW . . . 30

Week 1: Introduction to Text & Sequence Analytics

Fundamentals of Text Analytics

Text analytics, or **text mining**, is the process of extracting high-quality information from unstructured text. It differs from standard data processing because text is inherently 'noisy' and lacks a regular syntax or pattern.

- **Definition:** Transforming unstructured text into useful data through NLP, algorithms, and analytical methods.
- **The 3 Vs:** The vast amount of data created yearly (Volume, Velocity, Variety) necessitates modern storage like NoSQL and Vector Databases.
- **Core Tasks:**
  - Translation and Language Identification
  - Sentiment Analysis and Topic Modelling
  - Classification and Clustering

The DIKW Pyramid

This hierarchy illustrates how we move from raw facts to actionable insight. In exams, this is often used to explain the goal of information management vs. knowledge management.

Level	Description	Focus
Wisdom	Understanding 'Why'; used for high-level decision making	Know Why
Knowledge	Finding patterns, rules, and building predictive models	Know How
Information	Data that has been categorized, combined, and calculated	Know What
Data	Raw facts, figures, and measurements	Raw Input

Artificial Intelligence: Capabilities & Branches

AI is categorized by its 'reach' relative to human intelligence.

AI by Capability

- **Artificial Narrow Intelligence (ANI):** Specialized in one task (e.g., Alexa, Siri, or Generative AI).
- **Artificial General Intelligence (AGI):** Can perform any task a human can.
- **Artificial Super Intelligence (ASI):** Surpasses all human capabilities.

The Seven Branches (IEEE)

- **Machine Learning (ML):** Statistical models that learn from data.
- **Natural Language Processing (NLP):** Understanding and generating human language.
- **Neural Networks:** Models based on the human brain (interconnected nodes).
- **Computer Vision:** Interpreting visual or image data.
- **Robotics:** Programming machines for autonomous tasks.

- **Expert Systems:** Simulating human expertise in specific fields.
- **Fuzzy Logic:** Handling vagueness and uncertainty in reasoning.

Sequence Analysis & Bioinformatics

A unique aspect of this course is the crossover between human language and biological 'languages' like DNA.

- **Goal:** Analyzing DNA and protein sequences to discover biological insights.
- **The Connection:** Biological sequences use symbols (like English) where the order is the most critical factor, just like NLP.
- **Key Growth:** The volume of data in GenBank (Nucleotides, Amino Acids) has seen exponential growth over 30 years.

Historical Context & Thought Experiments

Dr. Healy highlights several milestones that defined the field:

- **Memex (1945):** Vannevar Bush's concept for 'associative memory' storage, the precursor to hypertext.
- **Turing Test (1950):** A test of whether a machine can 'imitate' a human well enough to fool an interrogator.
- **Searle's Chinese Room (1980):** A critique of 'Strong AI'. It argues that a machine can manipulate symbols (syntax) without ever understanding their meaning (semantics).
- **AI Winters:**
  - 1st AI Winter: Triggered by the 1966 ALPAC report.
  - 2nd AI Winter: Mid-1980s to early 1990s.

Modern Tools & Frameworks

To implement these theories, the industry uses specific Python and Java ecosystems.

- **General ML:** scikit-learn, PyTorch, TensorFlow, Keras.
- **NLP Specific:** NLTK, spaCy, Gensim.
- **LLM Ecosystem:** Hugging Face, LangChain, and models like GPT, Gemini, and Llama.
- **Data Interchange:** ONNX (Open Neural Network Exchange) allows models to move between different languages like Python and Java.

Week 2: Text Processing & Normalization

Text Representation

Computers do not process characters directly; they represent them as numeric values.

- **ASCII:** A basic numeric mapping for 128 characters, including control characters (like 'Line Feed') and standard Latin letters.
- **ISO-8859-1 (Latin 1):** An extension of ASCII standardized in 1987 to include 256 characters, covering most Latin and Greek alphabets.
- **Unicode:** The modern standard for world languages.
  - **UTF-8:** Uses 1 byte for basic characters; compatible with ISO-8859-1.
  - **UTF-16:** Uses 2 bytes (16 bits) per character, allowing for 65,536 characters, including ancient scripts like Ogham.

Java Strings

- Java uses UTF-16 internally for `char` and `String`.
- **Immutability:** Once a `String` is declared, it cannot be changed; modifying it actually creates a new object to improve performance.
- **String Pooling:** Literal strings (e.g., `String t = "Happy Days!"`) are shared in a 'String Pool' in the JVM heap to save memory.

Regular Expressions (Regex)

Developed in the 1950s, Regex is the 'de facto' standard for pattern matching and text manipulation.

Symbol	Meaning	Example
`.`	Any single character	`a.b` → `acb`
`\d`	Any digit (0-9)	`\d{4}` → `1970`
`\w`	Word character (a-z, 0-9)	`\w+` → `Hello123`
`^` / `$`	Start / End of string	`^Hi` / `end$`
`[^a-zA-Z]`	Match only letters	Useful for removing numbers or symbols

---

Text Normalization

Before analysis, text must be 'wrangled' or cleaned through a series of steps:

- **Tokenization:** Decomposing text into minimal meaningful units like words or sentences.
- **Morpheme Analysis:** Breaking words into stems and affixes (prefixes or suffixes) to find the 'base' meaning.
- **Stemming:** A heuristic 'affix stripper' that reduces a word to a root.
- **Lemmatization:** A more accurate, context-aware method that uses a dictionary (like WordNet) to find the valid base form (lemma).
- **Stop Word Removal:** Filtering out noisy, insignificant words (like 'the', 'and') that often make up 25%+ of a document.

Key Stemming Algorithms

Stemming is a trade-off between speed and accuracy.

- **Porter's Algorithm (1980):** The most widely used; classifies characters as Vowels (v) or Consonants (c) and applies a 5-step rule list.
- **Snowball Stemmer:** An updated version of Porter's algorithm supporting multiple languages.
- **Lancaster (Paice-Husk) Stemmer:** A 'greedy' iterative rules-based approach.

Common Issues:

- **Over-stemming:** Too aggressive; stems 'university' to 'universe'.
- **Under-stemming:** Too lazy; fails to stem 'knavish' to 'knave'.

Part of Speech (POS) Tagging

POS tagging labels words with lexical classes (noun, verb, etc.) based on their role in a sentence.

- **Penn Treebank:** The standard notation for tagging.
- **Common Tags:**
  - `NN`: Noun, singular
  - `VB`: Verb, base form
  - `JJ`: Adjective
  - `CD`: Cardinal number
  - `PRP`: Personal pronoun

Performance Metrics & Data Structures

- **Bloom Filter:** A probabilistic data structure used to test set membership efficiently; ideal for representing large sets of stop words or computing similarity metrics while saving memory.
- **Index Compression Factor (ICF):** Measures a stemmer's strength by the percentage of distinct words it reduces.

$$\text{ICF} = \frac{n - s}{n} \times 100$$

where n is the number of distinct words before stemming and s is the number after stemming.

Week 3: Tokenization & Sequence Segmentation

Text Tokenization Basics

- **Definition**: Segmenting text into meaningful atomic units (tokens) such as characters, sub-words, or words.

- **Vocabulary**: The complete set of unique tokens used by a specific tokenizer.

- **Out-of-Vocabulary (OOV)**: Words appearing in text that are missing from the tokenizer's pre-defined vocabulary.

Granularity Trade-offs

- **Word-level**: Often used for semantic analysis and Part-of-Speech tagging, but results in large vocabularies and frequent OOV misses.

- **Character-level**: Useful for spelling correction and OOV handling, but creates long sequences that are computationally expensive to process.

Sub-Word Tokenization

Decomposes words into smaller units to handle OOV words and provide better morphological understanding while keeping vocabulary size manageable.

Byte-Pair Encoding (BPE)

- **Concept**: A data compression algorithm adapted for tokenization by iteratively merging the most frequent adjacent pairs.

- **Models**: Used in GPT-2, RoBERTa, and XLM.

BPE Training Algorithm

1. Initialize vocabulary: $V = \text{Set of characters in } S$
2. **Do**:
   - Find: $(\text{Token}_{Left}, \text{Token}_{Right}) = \text{getMostFrequentAdjacentTokens}(S)$
   - Concatenate: $\text{Token}_{new} = \text{Token}_{Left} + \text{Token}_{Right}$
   - Update: $V = V + \text{Token}_{new}$
   - Replace all adjacent $(\text{Token}_{Left}, \text{Token}_{Right})$ in S with Token_{new}.
3. **Loop Until** `maxMerges` reached.

- **Complexity**:
  - Training time: $O(VC)$ where V is vocabulary size and C is corpus size.
  - Tokenization for text T: $O(T \log V)$

WordPiece

- **Concept**: Iterative sub-word tokenization similar to BPE but uses a likelihood scoring system for merges.

- **Models**: Primarily used by BERT.

Merging Score Formula

$$\text{score} = \frac{\text{freq}(\text{Token}_{Left}) \times \text{freq}(\text{Token}_{Right})}{\text{freq}(\text{Token}_{new})}$$

Markers

- Uses `##` to track sub-word positions (start, middle, or end) and preserve morphological structure.

Special Tokens

- `[CLS]`: Classification token at the start of every sequence.
- `[SEP]`: Separator for different text segments (e.g., Q&A).
- `[UNK]`: Unknown token for OOV words.
- `[PAD]`: Padding to reach fixed-size vector lengths.
- `[MASK]`: Used in BERT pre-training to hide words for prediction.

Unigram Tokenization

- **Concept**: A probabilistic algorithm that starts with a large vocabulary and iteratively removes tokens that cause the least log-likelihood loss.
- **Base Rule**: Base characters are never removed to ensure all strings remain tokenizable.

Log Loss Formula

$$\text{loss} = \sum_{i=1}^{n} \log \left(\sum_{w \in S(w_i)} p(w) \right)$$

Where $S(w_i)$ is the set of all possible tokenizations of a word w_i.

- **Training Complexity:** $O(nv)$ where n is training examples and v is vocabulary size.

Sequence Segmentation

Division of already tokenized text into smaller, overlapping sub-sequences, often using a sliding window.

N-grams

- **Definition**: A sequence of n contiguous characters.

- **Applications**: Spell checking, language modeling, and plagiarism detection.
- **Biological Context**: Called **k-mers** when used with DNA or protein sequences.
- **DNA Encoding**:
  - DNA sequences ($|\Sigma| = 4$) can be encoded with 2 bits:
    - A: 00
    - T: 01
    - G: 10
    - C: 11
  - A 64-bit long can encode a 32-mer.

Shingles

- **Definition**: A word or group of words, similar to n-grams but typically used for document-level analysis.
- **Applications**: Near-duplicate detection, document clustering, and data deduplication.

Skip-grams

- **Definition**: Similar to n-grams but allows gaps (skips) between words.
- **Value**: Captures semantic relationships between non-adjacent words, essential for creating dense word embeddings.

SentencePiece Framework

- **Origin**: Developed by Meta.

Features

- Implements optimized BPE, WordPiece, and Unigram (default).
- Reversible tokenization: (token id) $\leftrightarrow$ (Unicode)
- Excellent for languages without whitespaces (Chinese/Japanese) because it treats the whole sentence as a Unicode stream.
- Training time is reduced to $O(n \log n)$ using an **Enhanced Suffix Array (ESA)**.

Week 4: Sequence Similarity & Alignment

Mathematical Fundamentals of Similarity

To compare sequences, we distinguish between measuring how much they are alike versus how much they differ.

Similarity (s)

Measures the degree of correspondence between sequences. Higher similarity implies a closer relationship.

Distance (d)

Measures the number of changes required to transform one sequence into another.

Identity (I)

A Boolean indicator function:

$$I(a,b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{if } a \neq b \end{cases}$$

Metric Properties

For a distance function $d(x,y)$ to be a true metric, it must satisfy:

1. **Non-negativity:** $d(x,y) \geq 0$
2. **Identity of Indiscernibles:** $d(x,y) = 0 \iff x = y$
3. **Symmetry:** $d(x,y) = d(y,x)$
4. **Triangle Inequality:** $d(x,z) \leq d(x,y) + d(y,z)$

Distance Calculations

These quantify the cost of transforming one string into another.

Hamming Distance

The number of positions at which corresponding symbols differ.

Constraint: Sequences must be of equal length.

$$d_H(s_1, s_2) = \sum_{i=1}^{n} \mathbf{1}[s_{1i} \neq s_{2i}]$$

where $\mathbf{1}[\cdot]$ is the indicator function.

Levenshtein (Edit) Distance

The minimum number of insertions, deletions, or substitutions required to transform string A into string B.

Let $d(i, j)$ denote the distance between the first i characters of A and the first j characters of B.

Recurrence Relation:

$$d(i,j) = \min \begin{cases} d(i-1,j)+1 & \text{(Deletion)} \\ d(i,j-1)+1 & \text{(Insertion)} \\ d(i-1,j-1)+\text{cost} & \text{(Substitution)} \end{cases}$$

$$\text{where cost} = \begin{cases} 0 & \text{if } A[i] = B[j] \\ 1 & \text{if } A[i] \neq B[j] \end{cases}$$

Sequence Alignment

Alignment algorithms use a scoring matrix H and defined transition rules to find an optimal alignment.

Needleman-Wunsch (Global Alignment)

Aligns sequences from beginning to end. Suitable for closely related sequences of similar length.

Initialization: $H(i, 0) = i \times \text{gap}$ and $H(0, j) = j \times \text{gap}$

Recurrence Relation:

$$H(i,j) = \max \begin{cases} H(i-1,j-1) + S(a_i, b_j) & \text{(Match/Mismatch)} \\ H(i-1,j) + \text{gap} & \text{(Gap in B)} \\ H(i,j-1) + \text{gap} & \text{(Gap in A)} \end{cases}$$

Time complexity: $O(mn)$

Smith-Waterman (Local Alignment)

Finds the highest scoring local subsequences within two sequences.

Initialization: $H(i, 0) = 0$ and $H(0, j) = 0$

Recurrence Relation:

$$H(i,j) = \max \begin{cases} 0 & \text{(Restart)} \\ H(i-1,j-1) + S(a_i, b_j) & \text{(Match/Mismatch)} \\ H(i-1,j) + \text{gap} & \text{(Gap in B)} \\ H(i,j-1) + \text{gap} & \text{(Gap in A)} \end{cases}$$

The inclusion of 0 prevents negative scores and allows the alignment to restart at any position.

Advanced Scoring Parameters

Affine Gap Penalty

More realistic than a linear gap penalty because opening a gap is typically more costly than extending one.

$$W = g + (L-1)e$$

where:

- g = gap opening penalty
- e = gap extension penalty
- L = total length of the gap

BLAST Statistical Significance

BLAST evaluates whether an alignment is statistically meaningful.

Bit Score S'

A normalized alignment score independent of database size.

E-value (Expect Value)

The expected number of matches occurring by chance.

$$E = m \times n \times 2^{-S'}$$

where:

- m = query length
- n = database length
- S' = bit score

A very small or zero E-value indicates a statistically significant match.

Week 5: Alignment-Free Sequence Similarity

Fundamentals of Alignment-Free Similarity

Compare strings by token composition, ignoring positional information. Strings are represented as sets or bags of tokens (words, symbols, n-grams) rather than ordered sequences.

- $O(n)$ vs $O(n^2)$ for DP/alignment-based methods.
- Use **hashing** to convert variable-length tokens into fixed-length integers for ML pipelines.
- Can produce false positives when strings share composition but not structure, as positional information is lost.
- Token size (n-gram size) affects sensitivity: larger n-grams are more precise but sparser.
- Applications: plagiarism detection, text classification, clustering, genomics.

Measures vs. Metrics

A distance **measure** is not necessarily a **metric**. The four metric properties are covered in Week 4. The key point is that many alignment-free measures (e.g., Overlap coefficient) do not satisfy all four, so they are measures, not metrics.

Indices, Coefficients, and Distances

- **Coefficient:** A ratio-based scaling factor between sets.
- **Index:** A normalized similarity in $[0, 1]$, possibly combining multiple factors (e.g., Tversky Index of $0.8 = 80\%$ similar).
- **Distance:** Dissimilarity score where 0 implies identity.

Key Relationship: Similarity and distance are complementary. For any normalized similarity measure $\text{sim}(A, B)$, the corresponding distance is defined as:

$$d(A, B) = 1 - \text{sim}(A, B)$$

This applies to Cosine, Jaccard, Dice, Tversky, and Overlap measures.

Vector-Based Distances

Text is converted into a high-dimensional vector space using character frequencies, n-gram counts, or word embeddings. BOW structures ensure uniform-length vectors.

Cosine Similarity and Distance

Cosine of the angle between two n-dimensional vectors. Independent of magnitude. Used in transformers, clustering, and bioinformatics.

- **Cosine Similarity:**

$$\cos(s,t) = \frac{\sum_{i \in V} s_i t_i}{\sqrt{\sum_{i \in V} s_i^2}\sqrt{\sum_{i \in V} t_i^2}}$$

- **Cosine Distance:**

$$d_{cos}(s,t) = 1 - \cos(s,t)$$

The similarity value is bounded between 0 and 1: - $\cos = 0$ (90°): vectors are orthogonal (maximally dissimilar). - $\cos = 1$ (0°): vectors are parallel (maximally similar).

Token size matters. For two sentences with the same words in a different order:

Token Type	$\cos(A, B)$	Why
1-grams	≈ 1.0	Same word counts, order ignored
3-grams (word)	$= 0$	Phrases differ entirely
5-mers (char)	$= 0$	Character windows differ entirely

Euclidean Distance

Straight-line distance between two points. Sensitive to scale and has no notion of ordering: anagrams like *listen* and *silent* have identical character frequencies, so $d_E = 0$ despite different meanings.

$$d_E(s,t) = \sqrt{\sum_{i=1}^{n} (s_i - t_i)^2}$$

Manhattan Distance

Also called L_1, city block, or taxicab distance. Sums the absolute difference at each vector coordinate.

- **Manhattan Distance:**

$$d_M(s,t) = \sum_{i=1}^{n} |s_i - t_i|$$

Requires padding for strings of different lengths. Good when frequency differences matter more than order, e.g., spelling errors and anagram detection.

Canberra Distance

Weighted Manhattan where each difference is normalized by the sum of the values, making it less sensitive to scale. Range: $[0, n]$.

- **Canberra Distance:**

$$d_C(s,t) = \sum_{i \in V} \frac{|s_i - t_i|}{|s_i| + |t_i|}$$

Use when frequencies vary a lot in scale, or when small differences in small values are important. Prefer Manhattan when scale is consistent.

Chebyshev Distance

Also called L_∞ distance. Takes only the single largest difference between any coordinate, ignoring all others.

- **Chebyshev Distance:**

$$d_\infty(s,t) = \max_{i \in V} |s_i - t_i|$$

Use when the worst-case difference is what matters. Good for anomaly detection and KNN.

Minkowski Distance

Generalized distance that unifies Manhattan, Euclidean, and Chebyshev.

- **Minkowski Distance:**

$$d_p(s,t) = \left(\sum_{i \in V} |s_i - t_i|^p \right)^{\frac{1}{p}}$$

- $p = 1$ is equivalent to Manhattan distance.
- $p = 2$ is equivalent to Euclidean distance.
- $p = \infty$ is equivalent to Chebyshev distance.

Tune p to change sensitivity: higher p emphasizes the largest differences, at the cost of more computation.

Set-Based Similarities

These methods measure the similarity between sets of tokens derived from text.

Jaccard Similarity

Proposed by Paul Jaccard in 1901. Quantifies similarity by comparing shared elements against the total combined unique elements.

- **Jaccard Index:**

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

- **Jaccard Distance:**

$$d_J(A, B) = 1 - J(A, B)$$

Overlap Distance

Szymkiewicz-Simpson coefficient (1934/1960). Normalizes by the smaller set, making it useful for documents of very different sizes.

- **Overlap Coefficient:**

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

- **Overlap Distance:**

$$d_{\text{overlap}}(A, B) = 1 - \text{overlap}(A, B)$$

Sørensen-Dice Similarity

Weighted intersection, making it more sensitive to common elements. Smaller denominator penalizes mismatches less harshly than Jaccard. Better for short or variable-length texts.

- **Dice Index:**

$$\text{dice}(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

- **Dice Distance:**

$$d_{\text{dice}}(A, B) = 1 - \text{dice}(A, B)$$

Tversky Distance

An asymmetric similarity measure that quantifies the degree of overlap while explicitly weighting false positives and false negatives.

- **Tversky Index:**

$$\text{Tversky}(A, B) = \frac{|A \cap B|}{|A \cap B| + \alpha|A \setminus B| + \beta|B \setminus A|}$$

- **Tversky Distance:**

$$d_{\text{Tversky}}(A, B) = 1 - \text{Tversky}(A, B)$$

- Generalizes Jaccard when $\alpha = 1$ and $\beta = 1$.
- Generalizes Sørensen-Dice when $\alpha = 0.5$ and $\beta = 0.5$.

MinHash Algorithm

Developed by Andrei Broder (AltaVista, 1997). Represents each document as a signature of k minimum hash values from k different hash functions (using XOR and bit rotations). Scales Jaccard estimation to massive datasets.

- **MinHash Approximation:**

$$J(A, B) \approx \frac{n}{k}$$

- k is the total number of hash functions applied.
- n is the number of hash functions for which $h_{\min}(A) = h_{\min}(B)$.

The **expected value** of the MinHash similarity between two sets equals the Jaccard Index:

$$\mathbb{E}\left[\mathbf{1}[h_{\min}(A) = h_{\min}(B)]\right] = J(A, B)$$

MinHash Compatibility

Only works for set-based similarity measures.

Measure	Compatible?	Notes
Jaccard	Yes	Directly estimated
Sørensen-Dice	Yes (indirectly)	Derivable from Jaccard: $\text{Dice} = \frac{2J}{1+J}$ and $J = \frac{\text{Dice}}{2-\text{Dice}}$
Tversky	Yes (when $\alpha = \beta = 1$)	Reduces to Jaccard in this case
Overlap	No	Requires $\min(\lvert A\rvert, \lvert B\rvert)$, which MinHash cannot estimate
Cosine / Euclidean / Manhattan	No	These require vector arithmetic, which cannot be accurately estimated from hash signatures

Week 6: Text Feature Engineering

Feature Engineering Fundamentals

Before text can be processed by machine learning algorithms, it must be converted into a fixed-size input vector.

- Fixed-size inputs are ideal for continuous signals (e.g., sensors or game controllers), but text provides variable-length input.
- Encoding transforms variable-length text into a fixed format through mechanisms like n-grams, Huffman codes, or hashing.

Challenges of Using Text

- Context dependency dictates that meaning relies heavily on surrounding text and external knowledge.

- High-dimensional sparse vector spaces are generated by techniques like Bag of Words (BOW) or TF-IDF, leading to computational expense and the risk of overfitting.
- Linguistic nuances such as polysemy (multiple meanings for one word), synonymy (different words with similar meanings), and varying grammar rules complicate analysis.
- Extensive preprocessing is mandatory, including tokenization, lemmatization, stemming, and special character handling.

Data Sets and Data Frames

- A dataset is a general collection of data (structured, unstructured, or semi-structured).
- A data frame is a structured 2D matrix (rows + columns) used for ML, with defined data types.
- Within a data frame, continuous variables are typically normalized (rescaled between a minimum and maximum) or standardized (rescaled around a mean of zero and a standard deviation of one).
- Categorical or discrete variables are typically processed using one-hot encoding.

Text Encoding and Vectorization

Bag of Words (BOW)

- A multiset of tokens and their frequencies within a document.
- Can be extended to n-grams, shingles, or sub-words.
- **Limitation:** Loses word order $\rightarrow$ no contextual/semantic meaning.

Count Vectorization

- Creates a document-term matrix where each dimension corresponds to a specific token from the corpus.
- Values can represent raw frequencies, binary occurrences, or weighted measures.
- **Process:**
  1. Build a vocabulary (unique tokens across corpus)
  2. Create document-term matrix
  3. Encode each document as a vector

Shingles (n-grams)

- Help preserve local semantics and partial word order.

One-Hot Encoding

- Primarily used for categorical data rather than full text, this technique represents data as binary vectors where the vector length equals the total vocabulary size.

Term Frequency-Inverse Document Frequency

The TF-IDF model normalizes token frequencies by considering their relevance across the entire corpus, filtering out low-entropy "noise" words.

- **Term Frequency (TF):** Measures how frequently a term appears within a specific document relative to its length.

- **Term Frequency Formula:**

$$tf(t,d) = \frac{f(t,d)}{len(d)}$$

- **Inverse Document Frequency (IDF):** Measures the rarity and overall importance of a term across all documents.
- **Inverse Document Frequency Formula:**

$$idf(t,D) = \log\left(\frac{D}{n}\right)$$

- **TF-IDF Calculation:** The final weight is the product of TF and IDF, which highlights terms that are highly distinctive to a specific document.
- **TF-IDF Formula:**

$$tfidf(t,d,D) = \frac{f(t,d)}{len(d)} \times \log\left(\frac{D}{n}\right)$$

- TF-IDF reduces the weight of very common terms across documents (e.g., stop words) while increasing the relative importance of rarer, more informative tokens.

Okapi BM25

- A ranking function used in information retrieval systems (e.g. search engines)
- Balances:
  - Term frequency
  - Document length normalization
  - Term rarity (IDF)
- Short documents are boosted, long documents are penalized
- Long documents heavily penalize term weights in the denominator, while shorter documents increase the term weight.
- Parameter k_1 controls the scaling of term frequency, while b controls the strength of document length normalization.
- Computational complexity is historically $O(QC)$, but it is optimized in practice using inverted indices, document partitioning, and early termination heuristics.
- **BM25 Score:**

$$score(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

- **Okapi BM25 IDF:**

$$IDF(q_i) = \log\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right)$$

- **BM25 Optimizations:**

- Inverted indices (term → list of documents)
- Document partitioning (parallel processing)
- Early termination (stop when enough good results found)

Feature Selection and Hashing

Incremental Feature Selection

- A generate-and-test topology approach for neural networks to determine the optimal hidden layer configurations.
- **Hidden Layer Node Calculation:**

$$\text{Nodes} = \frac{|D|}{\alpha \times (N_{input} + N_{output})}$$

- $|D|$ = number of training samples, N_{input} = input neurons, N_{output} = output neurons, α = scaling factor (typically 2–10).

Vector Hashing

- Hashes variable-sized inputs (like n-grams) into a fixed-size vector array to drastically reduce vocabulary tracking.
- Converts variable-length input into fixed-length vectors
- Especially useful for text and high-dimensional data
- This is achieved by taking the modulus of the hash against the target array size.
- **Modulo Hash Index:** $index = hash(f)$ % n (where % is the *mod* operator)
- Particularly useful for domains with extremely large combinatorial feature spaces, such as biological sequences.
- For example, proteins built from 20 amino acids produce rapidly growing n-gram feature spaces:
  1-grams = 20, 2-grams = 400, 3-grams = 8,000, 4-grams = 160,000, 5-grams = 3,200,000.
- Hashing allows all such features to be compressed into a single fixed-length vector.

Dimensionality Reduction

To manage the sparsity and extreme dimensionality of text features, dimensionality reduction algorithms are commonly applied before classification.

- **Principal Component Analysis (PCA):** An unsupervised technique that projects data onto orthogonal axes to maximize variance.
- **Linear Discriminant Analysis (LDA):** A supervised technique that maximizes class separability.

Week 7: Introduction to Text Classification

Fundamentals of Text Classification

Text classification is the automated categorization of documents into predefined classes using machine learning models.

- **Applications:** Spam filtering, document organization, sentiment analysis, and topic modelling.
- **Model Types:** Classifiers can be supervised or unsupervised, and binary or multiclass.
- **Properties of Text Data:** Text is feature-rich and inherently high-dimensional.
- **Explainability (XAI):** Important but not always possible. Techniques like Shapley Values and LIME can help explain specific models, though some, like Neural Networks, remain black boxes.
- **Process Pipeline:**
  1. Preprocessing (cleaning and normalization).
  2. Feature extraction (BOW, TF-IDF, embeddings, hashing).
  3. Train/test split using a labeled corpus of documents.
  4. Model training and validation using standard ML statistical methods.
  5. Process, predict, and evaluate.

Supervised Classification Models

Different machine learning algorithms offer various trade-offs for text classification tasks.

Logistic Regression

A statistical model that predicts the probability of a binary outcome.

- **Advantages:** Highly interpretable (coefficients directly indicate feature importance), computationally efficient, and handles sparse high-dimensional data well.
- **Mechanism:** Applies the sigmoid function to a linear combination of features to output a probability between 0 and 1.
- **Logistic Function:**

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_n X_n)}}$$

Naive Bayes

A probabilistic classifier based on Bayes' theorem that assumes naive independence between features.

- **Advantages:** Very fast, simple, and serves as a strong computational baseline for text classification.
- **Mechanism:** Calculates probability based on word frequencies within a specific class. The independence assumption rarely holds true in natural language, but the model remains surprisingly effective.
- **Probability Equation:**

$$P(\text{doc}|\text{class}) = P(\text{word}_1|\text{class}) \times P(\text{word}_2|\text{class}) \times \cdots \times P(\text{word}_n|\text{class})$$

Support Vector Machines (SVM)

Developed in 1995, SVMs map data into a high-dimensional space to find an optimal separating hyperplane.

- **Mechanism:** Maximizes the margin between classes using support vectors (the specific data points closest to the hyperplane boundary).
- **Multiclass Handling:** Naturally a binary classifier, but handles multiclass environments using One-vs-Rest (OvR).
- **Advantages:** Highly effective for medium-sized datasets where a clear margin of separation exists, but computationally intensive for very large datasets.

Ensemble Methods & Random Forests

Combines multiple machine learning models into a single predictive model to improve overall accuracy, stability, and generalization capacity ("wisdom of crowds").

- **Mechanisms:**
  - **Bagging:** Reduces variance by training multiple models on random subsets of the training data with replacement.
  - **Boosting:** Reduces bias by building models sequentially, with each new model focusing on correcting the errors of the previous ones.
- **Random Forest:** An ensemble of decision trees where each tree is trained on a random data subset and makes an independent prediction. The forest's final prediction is determined by a majority vote. This significantly reduces the overfitting that individual trees are prone to.

Neural Networks (NN)

A mathematical model designed to simulate the structure and function of a biological neural network.

- **Structure:** Consists of highly interconnected layers of information-processing units called neurons.
- **Weights:** The knowledge of the network is stored in the weighted edges connecting nodes across layers. These weights are initialized randomly and updated during training.
- **Inputs and Outputs:**
  - All inputs must be converted into a fixed-sized feature vector before processing.
  - Each neuron receives multiple inputs, sums them, applies an activation function, and produces exactly one output.
  - A single neuron alone can only learn linearly separable tasks.

Activation Functions

Used by a neuron to compute its activation level based on inputs and weights. Activation functions should ideally be computationally simple (avoiding complex exponents) to maximize processing

speed during training iterations.

- **Primary Activation Functions:**
  - **Softmax:** Widely used in the output layer for multi-class classification. It transforms raw output scores (logits) into a probability distribution where all scores sum to 1. It pairs naturally with cross-entropy loss.
  - **Sigmoid:** A soft limiter operating smoothly within bounds of $[-4, 4]$ but optimally in $[-1, 1]$.
  - **Hyperbolic Tangent (Tanh):** A soft limiter that outputs values bounded between 0 and 1.
  - **ReLU (Rectified Linear Unit):** Computationally lightweight and heavily relied upon in deep learning.
  - **Leaky ReLU:** A variation of standard ReLU designed to prevent "dead" unrecoverable neurons.
- **Secondary Functions:** Step, Identity, LogSig, Gaussian, ELU, SELU, Softplus, Softsign, Swish, and Sinc.

Data Normalization

Normalizing data squashes inputs into a consistent, restricted range (typically $[-2, 2]$) to ensure network stability.

- **Importance:** Prevents extreme input values from immediately driving neuron outputs to absolute highs or lows, which is crucial for the stability of activation functions.
- **Application:** Normalization is strictly applied to the columns of a dataset, not the rows. It is generally unnecessary when using Tree-based ML models.
- **De-normalization:** Training a network on normalized data means it will produce normalized outputs, which must often be de-normalized for human interpretation.

Gradient Descent & Accelerated Learning

Gradient descent algorithms iteratively minimize training error by locating the lowest possible point on an error surface.

- **Learning Rate (α):**
  - Small α: Results in smaller weight adjustments, slower learning speeds, and a smooth, stable learning curve.
  - Large α: Accelerates learning drastically but risks inducing instability and oscillation within the network.
- **Momentum (β):** A parameter added to the delta rule to accelerate convergence, commonly set to $\beta \approx 0.95$.
- **Optimizers:** Categorized fundamentally into gradient-based systems (Momentum, Nesterov) and learning-rate-based systems (AdaGrad, RMSProp, Adam, AdaDelta).

Network Topology

The internal architecture of a neural network must be precisely tailored to the specific problem being solved.

- **Design Approach:** Topology design is empirical; there is no universal Standard Operating Procedure (SOP) that fits all datasets.
- **Node Allocation:**
  - Classification tasks require one output node (for binary) or multiple output nodes equal to the number of classes.
  - Regression tasks strictly require one output node.
- **Starvation vs. Saturation:**
  - **Starvation:** Occurs when a network possesses too many weights but receives insufficient training data to update them accurately.
  - **Saturation:** Occurs when a network receives massive amounts of data but lacks sufficient internal weights to model the complexity.

Unsupervised Learning (Clustering)

Algorithms designed to identify hidden patterns, relationships, or structures in unlabelled data without explicit external guidance.

K-Means Clustering

Developed by Stuart Lloyd in 1957, K-Means groups text documents into k distinct clusters based on content similarity.

- **Applications:** Topic modelling, large-scale document grouping, sentiment clustering, and language identification.
- **Mechanism:** Randomly places k initial centroids, assigns each document to the closest centroid, and iteratively recalculates the new centroid positions until the clusters stabilize.
- **Distance Metrics:** The distance is generally computed using standard Euclidean distance or normalized cosine distance.

Computing the Value of K

The scalar value of k dictates the precise number of clusters (or classification types) the algorithm will generate.

- **Heuristics:** Setting $k = 1$ merges all data into a single cluster; setting $k = |\text{Data Frame}|$ creates a dedicated cluster for every single point. A common baseline heuristic is $k = \frac{|\text{Data Frame}|}{2}$.
- **The Elbow Method:** A technique that involves incrementally increasing the value of k to measure the resulting Sum of Squared Errors (SSE).
- **SSE:** Represents the sum of the squared distances between a given centroid and all elements assigned to its specific cluster.
- **Inflection Point:** The optimal k (the "elbow") is the exact inflection point on the graph where increasing k further yields no significant mathematical reduction in the error rate.

Week 8: Introduction to Word Embeddings

Fundamentals of Word Embeddings

Word embeddings are dense, low-dimensional numerical representations of words. Unlike sparse methods (BOW, TF-IDF) that result in vectors the size of the vocabulary, embeddings represent words as fixed-size vectors of real numbers (typically 50 to 300 dimensions).

- **Semantic Encoding:** Captures the meaning of a word such that words used in similar contexts have similar vectors.
- **Similarity Computation:** Enables calculating similarity scores (e.g., Cosine Similarity) between words. It identifies not just synonyms, but also related concepts (e.g., "Galway" and "Ireland").
- **The 2013 Revolution:** The release of **Word2Vec** by Google transformed NLP by moving from discrete atomic symbols to continuous vector spaces.
- **Feature Representation:** A word embedding is a vector of numbers representing features of a word.
- **Fixed Vector Size:** Feature width is fixed (typically 50 to 300) and learned by a neural network.
- **Efficiency Gain:** Reduces comparison complexity from $O(n^2)$ to $O(n)$.
- **Storage:** Can be stored and queried efficiently in vector databases.
- **Latent Features:** Embedding dimensions capture hidden relationships that are not explicitly interpretable (unlike n-grams).

Static vs. Context-Sensitive Embeddings

- **Static Embeddings:** Every word has a single fixed vector regardless of context.
  - *Examples:* Word2Vec, GloVe, FastText.
  - *Limitation:* Cannot distinguish between polysemous words (e.g., "bank" of a river vs. "bank" for money).
- **Context-Sensitive Embeddings:** The vector for a word changes based on the surrounding words in a sentence.
  - *Examples:* BERT, ELMo, GPT.

Geometric Properties & Word Analogies

One of the most powerful features of word embeddings is that they capture linguistic relationships through vector arithmetic.

- **Analogy Logic:** The relationship between "Man" and "Woman" is geometrically similar to the relationship between "King" and "Queen".

- **Vector Arithmetic:**

$$V_{\text{King}} - V_{\text{Man}} + V_{\text{Woman}} \approx V_{\text{Queen}}$$

- **Distance Metrics:**
  - Cosine similarity (most common, range [-1, 1])
  - Euclidean distance
  - Manhattan distance
  - Hamming distance
  - Minkowski distance

One-Hot Encoding vs. Dense Embeddings

- **One-Hot Encoding:**
  - Vector size = Vocabulary size ($|V|$).
  - Categorical and sparse (mostly zeros).
  - **Weakness:** No notion of similarity. The dot product of any two distinct one-hot vectors is always 0.
- **Dense Embeddings:**
  - Vector size = Fixed hyperparameter (e.g., 100).
  - Continuous and dense.
  - **Strength:** Captures "closeness" in a high-dimensional manifold.

Word2Vec Architecture

Word2Vec is a predictive model that learns embeddings by trying to predict a word based on its neighbors (or vice versa).

- **Context Window:** A fixed-sized window of n words surrounding a target word.
- **Sliding Window:**
  - Moves across text to generate training data.
  - Produces (input, output) training pairs.
  - All words inside the window share the same context.
  - Window size controls context scope and number of training samples.
- **The Model:**
  - Input layer: One-hot encoded vector of size $|V|$.
  - Hidden layer: Linear projection (embedding layer).
  - Output layer: Softmax over vocabulary.
- **Weights as Embeddings:**
  - After training, the output layer is discarded.

  - The input $\rightarrow$ hidden weight matrix becomes the embedding lookup table.
- **Embedding Dimension:** Equal to the number of hidden layer units.
- **Scalability Issue:**
  - Weight matrix size = $|V| \times d$
  - Example: 31,218 words $\times$ 300 features $\approx$ 9.3 million weights
  - Makes training computationally expensive.

Week 9: Word2Vec Methods & Skip-Gram vs. CBOW

Continuous Bag of Words (CBOW)

The CBOW model predicts a **target word** based on its surrounding **context words**.

- **Goal:** Predict w_t given $\{w_{t-n}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+n}\}$.
- **Mechanism:**
  - Multiple context words are one-hot encoded.
  - Each maps to its embedding vector.
  - Embeddings are averaged into a single vector.
  - This vector is used to predict the target word.
- **Training Samples:**
  - Generates **one training sample per word**.
- **Efficiency:**
  - Faster to train than Skip-gram.
  - Works well for **frequent words**.
  - Captures **syntactic relationships** (e.g., drink, drinking, drinker).

Skip-Gram

The Skip-gram model does the inverse of CBOW: it uses a single **target word** to predict surrounding **context words**.

- **Goal:** Predict $\{w_{t-n}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+n}\}$ given w_t.
- **Training Samples:**
  - Generates **multiple samples per word**.
  - Number of samples $\propto$ window size.
- **Effectiveness:**
  - Better for **rare words**.
  - Works well with **small datasets**.
  - Produces more expressive embeddings.

- **Computation:**
  - More expensive than CBOW.

Mathematical Objective: Softmax and Cross-Entropy

The model aims to maximize the probability of the context word w_O given the center word w_I.

- **Softmax Function:**

$$P(w_O|w_I) = \frac{\exp(v'^{\top}_{w_O} v_{w_I})}{\sum_{w=1}^{V} \exp(v'^{\top}_{w} v_{w_I})}$$

- **Problem with Softmax:**
  - Requires computing probabilities over the entire vocabulary.
  - Time complexity: $O(V)$ per update.
  - Becomes infeasible for large vocabularies (millions of words).

Training Optimizations

To handle large vocabularies, Word2Vec uses approximation techniques:

Hierarchical Softmax

- Uses a binary (Huffman) tree.
- Reduces complexity from $O(V) \rightarrow O(\log V)$.

Negative Sampling (NEG)

- Replaces softmax with **sigmoid functions**.
- Converts problem into multiple binary classification tasks.
- Each output node acts as a **logistic regression classifier**.
- **Mechanism:**
  - Update:
    - 1 positive sample
    - ~5–20 negative samples
  - Only a small subset of weights updated per step.
- **Complexity:**
  - Reduced from $O(V) \rightarrow O(1)$ per update.

Subsampling

- Frequent words (e.g., “the”, “and”, “of”) are randomly discarded.
- **Motivation:**
  - These words generate too many redundant training samples.
- **Mechanism:**
  - Controlled by probability $P(w_i)$.
  - Typical threshold: 0.001.
- **Effect:**
  - Reduces training size
  - Speeds up training
  - Improves embedding quality

Context Position Weighting

- Words closer to the target word are more important.
- **Implementation:**
  - Randomly shrink the context window size.
  - Words near the center are more likely to remain.
  - Words further away are more likely to be dropped.
- **Result:**
  - Implicit weighting without explicit weights.

Skip-Gram vs. CBOW

- **CBOW:**
  - Faster and more efficient.
  - Produces fewer training samples.
  - Better for **frequent words**.
  - Better at **syntactic relationships**.
- **Skip-Gram:**
  - Slower but more expressive.
  - Generates more training samples.
  - Better for **rare words**.
  - Performs better on **small datasets**.
  - Larger window $\rightarrow$ more training data.
